

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 建立 Rust 内建类型全图:整数 (i8-i128)、浮点 (f32/f64)、bool、char、元组 ()——并与 C 的 char/int/long/float/double ——对照
- 掌握 **let 绑定**:默认不可变、mut 可变、遮蔽(shadowing),并理解"Rust 没有未初始化变量"这条与 C 截然不同的规定
- 熟练写出各类字面量:进制前缀、下划线分隔符、类型后缀、字节字面量、raw 字符串
- 掌握 **const** 与 **static** 的语义,与 C 的 #define/const 对照
- 理解"Rust 没有隐式类型转换":as 的截断语义与 TryFrom/TryInto 的安全转换
- 分清 String 与 &str 两种字符串,知道"长度"与"字符数"的区别(UTF-8)
- 入门数组、元组、Vec、强类型枚举与 type 别名;埋下伏笔:Rust 的类型系统里带着"所有权"信息 (第5章展开)

💡 从 C 到 Rust:本章主题如何迁移

C 里一个 int 是多少字节?标准只保证"至少 16 位",在不同平台上可能是 16、32 或 64 位;而 Rust 的类型名直接把位宽写在名字里: i32 就一定是 32 位有符号整数。位宽、有无符号、字节序一目了然,这是 Rust 给 C 程序员的第一份礼物——类型自述其大小。

C 的局部变量不初始化就读,是未定义行为(UB),它可能打印出任何垃圾值;Rust 直接**禁止**未初始化变量存在——let x; 之后不使用、或未赋初值就使用,都是编译错误。把 UB 变成编译错误,是贯穿全书的主线:第5章所有权、第9章并发安全,走的都是同一条路。

C 的类型转换是"隐式 + 悄悄截断": int x = 300; char c = x; 得到一个 44,编译器最多给个警告。Rust 的态度是**零隐式转换**:所有转换必须写出来, as 负责"我就是要截断", TryFrom 负责"可能失败请告诉我"。类型转换从此成为显式决策,而不是潜伏的 bug。

Rust 的 String 之于 C 的 char[],相当于"自带长度的动态字符数组":不需要 strlen 数结尾、不需要 strcpy 防溢出、不需要担心 strcat 写穿缓冲区——内存安全是编译器默认行为,不是程序员的责任。

最后请记住本章最关键的一句伏笔: Rust 的每个类型不仅描述"值的形状",还描述"值的所有权归属"。比如 String 拥有自己的数据,而 &str 只是借用别人的数据。第5章将把这个"带所有权的类型系统"彻底讲透——你现在看到的每一条规则,都是在为那一刻铺路。

📦 前置知识自查

- C 的基本类型:char/short/int/long/float/double 及其字节数与范围
- C 的变量声明与初始化、#define 与 const 的差别
- C 的整型/浮点字面量、转义序列(\n \\ \")
- C 的强制转换 (int)x 与隐式转换(char c = 300 会发生什么)
- C 的字符数组 char s[] = "abc"、strlen / strcpy 的用法
- C 的数组、struct、enum、typedef

本章学习路线

1. **第一阶段(约1.5小时,建立直觉)**:通读 §1.1–§1.3,把"类型表"与"let 绑定"对照 C 记住,随手改改示例代码
2. **第二阶段(约1小时,攻转换)**:重点做 §1.4 的转换练习,把 C 的"隐式"习惯彻底改掉
3. **第三阶段(约1小时,练字符串与复合类型)**:§1.5–§1.7,亲手写字符串拼接与 Vec 增删
4. **验收标准**:不看资料,能背出类型表、写出 10 种字面量、解释"为什么没有未初始化变量"

本章知识导图

- §1.1 类型总览:与 C 类型——对照
 - 整数家族 i8–i128 / 浮点 f32/f64 / bool / char / 单元类型 ()
- §1.2 变量绑定:let、不可变默认、mut、遮蔽、无未初始化
- §1.3 字面量与常量:进制、分隔符、后缀、const 与 static
- §1.4 类型转换:零隐式转换、as 截断、TryFrom/TryInto
- §1.5 字符串:&str 与 String、UTF-8 长度 vs 字符数
- §1.6 复合类型入门:数组、元组、Vec、切片
- §1.7 枚举与类型别名:强类型枚举、带值枚举、type
- 速查表 → 最小必会清单 → 自测题 → 章末实验

§1.1 类型总览:与 C 类型——对照

1.1.1 整数家族:类型名自带位宽

C 的 `int` 位宽依赖平台, `long` 在 64 位 Linux 是 8 字节、在 Windows 是 4 字节——跨平台代码里这是反复踩坑的根源。Rust 把"符号 + 位宽"直接写进类型名, **类型与平台无关**:

Rust 类型	C 对应	字节数	取值范围
i8	signed char	1	-128 ~ 127
i16	short	2	-32768 ~ 32767
i32	int	4	±21 亿
i64	long long	8	±9.2×10 ¹⁸
i128	(C 无)	16	超大整数
u8	unsigned char	1	0 ~ 255
u16/u32/u64/u128	unsigned short/int/long long	2/4/8/16	0 ~ 2 ⁿ -1
isize / usize	ptrdiff_t / size_t	8(64 位平台)	与指针同宽
f32	float	4	±3.4×10 ³⁸
f64	double	8	±1.8×10 ³⁰⁸
bool	_Bool / int	1	true / false
char	char(单字节)	4	Unicode 标量值
()	void(占位)	0	只有一个值

对照要点: `isize/usize` 对应 C 的 `size_t`, 用于长度、下标、指针算术结果; `char` 是 4 字节的 Unicode 字符, 与 C 的 1 字节 `char` 完全不同, 字符串用专门的 `String / &str` (§1.5); `bool` 是真正的布尔类型, Rust 没有 C 那种“非零即真”的隐式真值转换。单元类型 `()` 表示“没有任何值”, 相当于 C 里 `void` 返回值的位置。

示例 1-1 观察各类型的大小

```
fn main() {
    // std::mem::size_of 返回类型在内存中占用的字节数
    println!("i8 : {} 字节", std::mem::size_of::<i8>());
    println!("i32 : {} 字节", std::mem::size_of::<i32>());
    println!("u64 : {} 字节", std::mem::size_of::<u64>());
    println!("usize: {} 字节", std::mem::size_of::<usize>());
    println!("f64 : {} 字节", std::mem::size_of::<f64>());
    println!("bool : {} 字节", std::mem::size_of::<bool>());
    println!("char : {} 字节", std::mem::size_of::<char>());
    // 引用类型(指针)在 64 位平台上是 8 字节
    println!("&i32: {} 字节", std::mem::size_of::<&i32>());
    println!("() : {} 字节", std::mem::size_of::<()>());
}
```

1.1.2 布尔类型: C 没有真正的 bool

C 的 `_Bool` (或 `int`) 与整数互通: `if (x)`、`return 0`; 到处是隐式转换。Rust 的 `bool` 只有 `true / false` 两个值, 不能与整数互相转换——`if 1 { }` 是编译错误。好处是: 条件表达式永远只表达“真假”这一件事, 没有 C 里 `if (ptr)`、`if (fd)` 那种依赖约定俗成的写法。

1.1.3 字符类型与编码

C 的 `char` 是 1 字节,存 ASCII 或按地区编码(GBK 等)凑合;Rust 的 `char` 是 4 字节的 Unicode 标量值, '你'、'A'、'😄' 都是合法 `char`。写 Unicode 转义: `'\u{4F60}'` 就是 '你'。字符串默认 UTF-8 编码,每个汉字 3 字节,这与 C 的"一个字符一个字节"的直觉完全不同——§1.5 会专门讲"字节数 ≠ 字符数"。

§1.2 变量绑定:let 与不可变性

1.2.1 声明与初始化:没有"未初始化"状态

C 的 `int x;` 之后直接读 `x`,是 UB:局部变量里是栈上的垃圾值。Rust 的规则是 变量必须在使用前初始化,否则编译不过:

```
fn main() {
    let x = 42;           // 整数默认推导为 i32
    let y: i64 = 42;      // 显式类型注解
    let mut count = 0;    // mut 关键字声明"可变的绑定"
    count += 1;           // 只有 mut 绑定才能被修改

    // let z;             // 编译错误:未初始化就声明
    // println!("{z}");    // 且无法使用未初始化变量
    println!("x = {x}, y = {y}, count = {count}");
}
```

注意 `let` 声明的是**绑定**(binding),不是 C 意义上的"变量":`let` 把名字绑到一个值上,默认不可变。C 的变量默认可**变**,`const` 需要显式加;Rust 恰恰相反:默认不可变,`mut` 需要显式加。这个"反过来的默认值"是 Rust 最重要的文化之一:编译器默认帮你锁住数据,只有你明确要求才放开。

1.2.2 遮蔽(shadowing):同名新绑定

C 里同名变量在同一个作用域内不能重新声明;Rust 允许 `let` 再次声明同名绑定,新绑定"遮蔽"旧的——旧值不会消失,只是这个名字不再指向它。遮蔽常用于"对同一个值做加工"的链式写法,且**可以在遮蔽时改变类型**:

```
fn main() {
    let name = "zhangsan"; // &str 类型
    let name = name.to_uppercase(); // 遮蔽:现在 name 是 String
    let name = name.len();   // 再遮蔽:现在 name 是 usize
    println!("{name}");      // 9(字节数)

    // 遮蔽与 mut 的区别:遮蔽创建新绑定,旧绑定仍存在
    let x = 5;
    let x = x + 1;           // 新绑定 x = 6
    let x = x * 2;           // 新绑定 x = 12
    println!("{x}");
}
```

⚠ 易错点 1:以为 Rust 也有 C 的"零值默认"或"未初始化垃圾值"

两个都没有。`let x: i32;` 不赋初值就使用,是**编译错误**;全局 `static` 虽然像 C 全局变量一样有零值(0、0.0、false),但普通局部变量绝不自动清零,也绝不允许读未初始化内存。C 的"忘记初始化 → 读到垃圾值"这一类 bug,在 Rust 里被编译器直接消灭。

1.2.3 与 C 的声明对照

C 的写法	Rust 的写法	说明
<code>int x = 5;</code>	<code>let x: i32 = 5; 或 let x = 5;</code>	Rust 可省略类型(推导)
<code>const int N = 10;</code>	<code>const N: i32 = 10;</code>	Rust 的 <code>const</code> 必须注解类型
<code>int *p = NULL;</code>	<code>let p: Option<i32> = None;</code>	Rust 没有 NULL,用 <code>Option</code> (第14章)
<code>char buf[100];</code>	<code>let buf = [0u8; 100];</code>	数组字面量加类型后缀
<code>int x; /* 读它=UB */</code>	<code>let x: i32; /* 编译就拒绝 */</code>	无未初始化状态

§1.3 字面量与常量

1.3.1 整数字面量:进制、分隔符、后缀

Rust 支持 C 的全部进制写法(0x/0o/0b 与十进制),并新增 **下划线数字分隔符** 提升可读性。整数默认 `i32`;需要其他类型时,要么注解,要么写类型后缀:

```
fn main() {
    let a = 255;           // 十进制,默认 i32
    let b = 0xFF;          // 十六进制:255
    let c = 0o377;         // 八进制:255
    let d = 0b1111_1111;   // 二进制:255,下划线只作分隔
    let e = 1_000_000u64;  // 后缀 u64:一百万
    let f = 3.14f32;       // 浮点后缀 f32(默认是 f64)
    let g = 1.2e-3;        // 科学计数法:0.0012
    let h = 0xFFu8;        // 后缀指定无符号小类型
    println!("{a} {b} {c} {d} {e} {f} {g} {h}");
}
```

1.3.2 字符、字节与字符串字面量

```
fn main() {
    let byte = b'A';       // 字节字面量:u8 类型的 65(C 里就是 char)
    let ch = '你';         // char 类型:Unicode 字符
    let esc = '\n';        // 转义序列与 C 相同
    let unicode = '\u{4F60}'; // 按码点写:'你'

    let s = "hello";       // &str:不可变字符串切片
    let raw = r"c:\new\dir"; // raw 字符串:不解析转义,反斜杠原样
    let raw2 = r#"含"引号"的文本"#; // 加 # 包裹,内部可含双引号
    let bytes = b"abc";    // 字节数组:&[u8; 3]
    println!("{byte} {ch} {esc:?} {unicode} {s} {raw} {raw2} {:?}", bytes);
}
```

`r"..."` raw 字符串对 C 程序员是福音:写 Windows 路径、正则表达式时不再需要把每个反斜杠写成 `\\`。需要内部出现双引号时,用 `r#"..."#` (引号数可增减)。

1.3.3 const 与 static:与 C 的对照

C 的 `#define` 是文本替换, `const` 只是"只读变量"; Rust 的 `const` 是真正的编译期常量:值必须在编译期求出来,且每个使用点都会被内联。C 里" `const int` 不能做数组长度"的著名限制,在 Rust 中不存在——`const N: usize = 8;` 直接可用于数组长度。若需要"有固定内存地址的全局变量"(对应 C 的全局变量),用 `static`:

```
// 编译期常量:名字大写、必须注解类型
const MAX_PLAYERS: u32 = 8;
const FONT_SIZE: f64 = 12.0;
const GREETING: &str = "欢迎回来";

// 全局静态变量:有固定地址,生命周期为 'static'
static APP_NAME: &str = "成绩管理系统";

fn main() {
    println!("{APP_NAME} 支持最多 {MAX_PLAYERS} 人,字号 {FONT_SIZE}");
    println!("{GREETING}");
    // 数组长度直接用 const,这在 C 里做不到(除非宏)
    let buffer: [u8; MAX_PLAYERS as usize] = [0; 8];
    println!("buffer 长度:{}, buffer.len()");
}
```

★ 重点:const 与 let 的分工

`const` 用于"编译期就确定的常量"; `let` 用于"运行时才有的变量"——即使 `let x = 42` 也是运行时的绑定。`const` 没有类型推导,必须注解类型;命名约定全大写。C 程序员最容易犯的错:把 `const` 当"只读变量"到处用——Rust 里编译器会告诉你哪些值能进 `const`,进不去的就用 `let`。

§1.4 类型转换

1.4.1 零隐式转换

C 里 `int + double`、`char c = 300`、`if (ptr)` 到处都是隐式转换与截断。Rust 的规则只有一条:不写转换,就没有转换。类型不同的值之间做运算、赋值、比较,一律编译错误——这让"类型错误"在编译期全部暴露,而不是在运行期悄悄变出垃圾值。

```
fn main() {
    let x: i32 = 42;
    // let y: i64 = x;          // 编译错误!i32 不会自动变 i64
    let y: i64 = x as i64;     // as:显式转换,窄→宽,安全
    println!("{y}");

    // 混合运算也必须显式转换
    let a: u32 = 10;
    let b: u64 = 5;
    // let c = a + b;          // 编译错误
    let c = a as u64 + b;     // 先把 a 转成 u64 再相加
    println!("{c}");
}
```

1.4.2 as 的截断语义:与 C 的强制转换一致

`as` 执行的是**底层位级转换**:宽→窄直接截断高位,浮点→整数向零截断。这与 C 的强制转换语义一致——区别在于 Rust 让你把"我知道会截断"写在明面上:

```
fn main() {
    let big: i32 = 300;
    let small: u8 = big as u8;    // 300 % 256 = 44,高位被丢弃
    println!("{}", small);      // 44

    let pi: f64 = 3.99;
    let n: i32 = pi as i32;      // 向零截断:3(不是四舍五入)
    println!("{}", n);

    // 负数转无符号:按位补码语义(与 C 一致)
    let m: i32 = -1;
    let u: u32 = m as u32;      // 4294967295
    println!("{}", u);

    // bool 与整数互转必须显式
    let flag: bool = true;
    let one: u8 = flag as u8;    // 1
    println!("{}", one);
}
```

1.4.3 TryFrom/TryInto:可能失败的转换

`as` 永远成功,但"300 放不进 u8"这种问题它不告诉你。当你想要"成功就取新值、失败就报错"的语义时,用 `TryFrom / TryInto` ——返回 `Result`,第9章会系统讲错误处理,这里先见个面:

```
fn main() {
    let big: i64 = 300;
    match i8::try_from(big) {
        Ok(v) => println!("转换成功:{v}"),
        Err(e) => println!("转换失败:{e}"), // 300 超出 i8 范围
    }

    let x: u32 = 42;
    let x_u8: u8 = x.try_into().unwrap_or(0); // 失败时给默认值 0
    println!("{}", x_u8);
}
```

⚠ 易错点 2:把 C 的"自动转换"习惯带进 Rust

最常见的三类编译错误:① 函数实参类型不匹配——C 里 `int` 传进 `double` 参数自动升格,Rust 必须 `as f64`;② 下标与长度混用——`i32` 做下标,但 `vec.len()` 是 `usize`,得用 `i as usize`;③ 字符与整数混用——`'0'` `as u8` 才等于 48,直接 `'0'` 是 `char` 不是数。见到 "mismatched types" 报错,第一反应就是:补一个显式转换。

§1.5 字符串:&str 与 String

1.5.1 两种字符串:C 的 char[] 一分为二

C 的字符串就是 `char[]` + 结尾 `'\0'`, 没有"字符串"这个类型。Rust 把字符串拆成两个类型:**String**(拥有数据、可增长、可修改)与 **&str**(借用一段文本、不可修改)。字符串字面量 `"abc"` 的类型是 `&str`, 相当于 C 的 `const char*` 字面量——但它自带长度, 不需要 `strlen`:

```
fn main() {
    // &str: 借用了一段文本, 不能修改
    let greeting: &str = "你好, Rust";
    println!("{greeting}");

    // String: 拥有文本的所有权, 可以修改、增长
    let mut name = String::from("张");
    name.push_str("三");           // 追加字符串
    name.push('!');               // 追加单个字符
    println!("{name}");           // 张三!

    // &str 与 String 的转换(这两个转换经常被调用, 免费)
    let s: String = String::from("hello");
    let view: &str = &s;          // String -> &str: 取引用即可
    let owned: String = view.to_string(); // &str -> String: 复制一份
    println!("{view} {owned}");
}
```

1.5.2 长度 vs 字符数: UTF-8 的第一课

`str.len()` 返回**字节数**, `str.chars().count()` 返回**字符数**。因为 UTF-8 里汉字占 3 字节, "你好, Rust" 是 7 个字符、11 个字节。C 程序员习惯"下标访问 = 取字符", Rust 里 `s[1]` 是**编译错误**——必须 `chars().nth(1)`, 而且它是 $O(n)$ 。这是 Rust 对"字符串是字节数组"的诚实: 底层就是字节, 想要字符得明确转换:

```
fn main() {
    let s = "你好, Rust";
    println!("字节数: {}", s.len());           // 11
    println!("字符数: {}", s.chars().count()); // 7

    // 按字节遍历(快, 但看到的是 UTF-8 字节)
    for b in s.bytes().take(3) {
        println!("字节: 0x{:02X}", b);
    }
    // 按字符遍历(正确, 但更慢)
    for ch in s.chars().take(2) {
        println!("字符: {ch}");
    }
    // 注意: 不能像 C 那样 s[0] 取字符——按字节下标会切到半个汉字
}
```

1.5.3 常用 API 速查

```
fn main() {
    // 拼接:format! 最常用,不产生所有权问题
    let a = "hello".to_string();
    let b = "world".to_string();
    let msg = format!("{a}, {b}!");
    println!("{msg}");

    // 查找、截取(按字节范围,务必用字符边界)
    let text = "Rust 语言";
    if let Some(pos) = text.find("语言") {
        println!("找到位置:{pos}"); // 5(字节偏移)
    }
    let sub = &text[..4]; // 取前 4 字节:"Rust"
    println!("{sub}");

    // 替换、拆分、大小写
    let t2 = text.replace("语言", "编程语言");
    println!("{t2}");
    for part in "a,b,c".split(',') {
        print!("{part} ");
    }
    println();
    println!("{}", "rust".to_uppercase()); // RUST
}
```

⚠ 易错点 3:用 `len()` 当"字符数",或按字节下标切字符串

"你好".`len()` 是 6 不是 2。`len()` 返回字节数;要字符数必须 `.chars().count()`。更危险的是 `&s[0..1]` 这种字节切片:如果切在汉字中间(UTF-8 多字节字符的内部),会直接 `panic`(第9章讲 `panic`)——C 里你顶多切出一个乱码,至少不崩。这也是为什么 Rust 的字符串下标访问干脆被禁用。

§1.6 复合类型入门:数组、元组、Vec

1.6.1 数组:固定长度,类型 `[T; N]`

Rust 数组把"长度"写进类型: `[i32; 5]` 表示 5 个 `i32`。长度 `N` 是编译期常量,这与 C 的 `int a[5]` 类似,但 Rust 数组越界访问是 `panic`(可预期),不是 C 那种静默内存破坏:

```
fn main() {
    let scores: [i32; 5] = [90, 85, 78, 92, 88];
    let zeros = [0u8; 100]; // 100 个 0(类似 C 的 memset 后)
    println!("第一个:{}", scores[0]);
    println!("长度:{}", scores.len()); // len() 返回 usize
    // println!("{}", scores[10]); // 越界:运行期 panic,进程退出

    // 数组默认也是"复制"语义—for 遍历逐个拷贝
    for s in scores {
        print!("{}", s);
    }
    println!();
}
```

1.6.2 元组:把不同类型的值打包

元组把多个(可以不同类型的)值合成一个,对应 C 里"为返回多个值而定义的临时 struct",但更轻量。用下标 `t.0` 或解构访问:

```
fn main() {
    // (String, u8, f64):一个学生的三种信息
    let student = (String::from("张三"), 21, 87.5);
    println!("姓名:{}, 年龄:{}, 成绩:{}", student.0, student.1, student.2);

    // 解构:一次拆开
    let (name, age, score) = student;
    println!("{name} 明年 {age} 岁,目标 {score} 分以上");
    // 解构后原元组仍可用(三个字段都是可复制类型时);此处已 move,见第5章
}
```

1.6.3 Vec:动态数组(向量)

`Vec<T>` 是 Rust 的标准动态数组,对应 C 里"自己 malloc 出来的数组 + 手动记录长度"。它能按需增长,离开作用域自动释放——不需要 free,不需要 realloc,不需要防越界:

```
fn main() {
    let mut nums: Vec<i32> = Vec::new();
    nums.push(10); // 末尾追加
    nums.push(20);
    nums.push(30);
    nums.pop(); // 末尾弹出:30

    let v = vec![1, 2, 3, 4, 5]; // vec! 宏快速构造
    println!("长度:{}, 第3个:{}", v.len(), v[2]);

    // 遍历(第3章细讲 for;此处先感受)
    let mut sum = 0;
    for n in &v {
        sum += n;
    }
    println!("总和:{}", sum);
}
```

1.6.4 切片:看数组的一段

`&[T]` 切片是"指向一段连续元素 + 长度"的视图,对应 C 的"指针 + 长度"惯用法,但 长度随视图走,越界由编译器和运行期检查兜底。对 `Vec` 和数组都能取切片:

```
fn main() {
    let v = vec![1, 2, 3, 4, 5, 6];
    let slice: &[i32] = &v[1..4];    // 视图:第2到第4个元素
    println!("切片长度:{}, 首元素:{}, slice.len(), slice[0]");

    // 数组自动支持切片语法
    let arr = [10, 20, 30, 40];
    let head: &[i32] = &arr[..2];    // 省略起点:从头开始
    println!("{head:?}");
}
```

§1.7 枚举与类型别名

1.7.1 强类型枚举:C 的 `enum` 只是整数的"马甲"

C 的 `enum` 本质是 `int`:可以赋任意整数、可以加减、可以和其他枚举互相隐式转换。Rust 的枚举是**独立类型**:不能当整数用、不能与别的枚举混用、也没有 C 那种"默认 0 开始递增"的自动整数值:

```
enum Direction {
    North,
    East,
    South,
    West,
}

fn main() {
    let dir = Direction::North;
    // let n: i32 = dir;        // 编译错误!枚举不能隐式转整数
    // let d = Direction::North + 1; // 编译错误!没有加法

    // 需要"枚举对应的显示文本"时,用 match(第3章)显式映射
    let text = match dir {
        Direction::North => "北",
        Direction::East  => "东",
        Direction::South  => "南",
        Direction::West   => "西",
    };
    println!("方向:{text}");
}
```

1.7.2 带值枚举:每个变体可携带数据(第14章的主角)

Rust 枚举的变体可以各自携带数据——这就是"和类型"(sum type)的雏形, `Option` / `Result` 都是它。C 里要模拟这种"要么这个要么那个还带数据",只能靠 `union` + 标志位,既笨拙又不安全:

```
// 消息:要么退出,要么移动(带坐标),要么写文本(带字符串)
enum Message {
    Quit,                                // 不带数据
    Move { x: i32, y: i32 },             // 带两个字段
    Write(String),                       // 带一个 String
}

fn main() {
    let m = Message::Write(String::from("保存成功"));
    match m {
        Message::Quit => println!("退出"),
        Message::Move { x, y } => println!("移动到 ({x}, {y})"),
        Message::Write(text) => println!("写入:{text}"),
    }
}
```

1.7.3 type 别名:给类型起短名字(不产生新类型)

对应 C 的 `typedef`。别名与原名完全等价,编译器不做区分——想要"防混淆的新类型",要用第14章的 `newtype` 模式:

```
type Meters = f64;                      // 别名的用途:缩短长类型名、表达意图
type Callback = fn(i32) -> i32;        // 函数指针类型别名

fn main() {
    let distance: Meters = 100.5;      // 本质还是 f64
    println!("{distance}");
}
```

★ 本章伏笔:类型系统里带着所有权

回顾本章出现的类型: `String` 拥有数据, `&str` 借用数据; `Vec<T>` 拥有堆上的缓冲; `Box<T>` (第5章)拥有堆上的单块数据。也就是说, Rust 的类型天生携带"谁拥有这块内存"的信息——这是 C 的类型系统(指针只是地址,不关心归属)永远表达不了的一维。第5章我们会看到:正是这多出来的一维,让 `use-after-free`、悬垂指针、数据竞争从"运行期事故"变成"编译期错误"。

本章速查表

主题	结论 / 写法
整数类型	i8/i16/i32/i64/i128, u8/u16/u32/u64/u128, isize/usize;默认推导 i32
浮点 / 布尔	f32 / f64(默认);bool 只有 true/false,不与整数互通
字符	char 为 4 字节 Unicode; 'A'、'\u{4F60}';字节字面量 b'A' 是 u8
变量绑定	let x = 5; 不可变; let mut x = 5; 可变; let x: i32 = 5; 注解类型
未初始化	不存在;使用未初始化变量是编译错误(C 是 UB)
遮蔽	let x = x + 1; 创建同名新绑定,可换类型
整数字面量	0x 十六 / 0o 八进 / 0b 二进 / 1_000_000 分隔符 / 42u8 后缀
raw 字符串	r"..." 不解析转义; r#"..."# 内部可含双引号
常量	const N: u32 = 8; 编译期常量,必须注解类型; static 有固定地址
类型转换	无隐式转换; as 截断语义; TryFrom / try_into() 安全转换
字符串	String 可增可改; &str 只读借用; len() 是字节数, chars().count() 是字符数
拼接/格式化	format!("{a}, {b}"), push_str、to_string()
数组 / 元组	[T; N] 定长; (a, b, c) 异类打包,解构取出
Vec	vec![...], push/pop/len/下标;越界 panic
切片	&v[1..4] 视图,自带长度
枚举 / 别名	强类型,不可当整数;变体可带数据; type X = ...

最小必会清单

- 能背出整数/浮点/布尔/字符类型表,并说出与 C 每个类型的对应与差异
- 能解释"为什么 Rust 没有未初始化变量",以及 mut 与遮蔽的区别
- 能写出 6 种以上字面量:十进制、十六进制、八进制、二进制、分隔符、后缀、raw 字符串
- 能说明 const 与 static、与 C 的 #define 的区别
- 能解释"为什么 Rust 禁止隐式类型转换",并正确使用 as 与 try_into()
- 能说出 String 与 &str 的区别,以及"为什么 s[0] 不能取字符"
- 能说出"你好"的 len() 与 chars().count() 分别是多少
- 能用数组、元组、Vec 各写一个例子,并说明越界访问的后果
- 能说明 Rust 枚举与 C 枚举的三个本质区别
- 能说出"类型系统里带着所有权"这句话的含义(给一个例子)

自测题

Q 1. C 的 `int` 与 Rust 的 `i32` 有什么本质区别? `long` 与 `i64` 呢?

✅ 参考答案

C 的 `int` 只保证"至少 16 位",实际位宽随平台; `long` 在 64 位 Linux 是 8 字节、Windows 是 4 字节。Rust 的 `i32` 名字即规格:在任何平台都是 32 位有符号。 `long` 对应不到 Rust 的任何一个类型——需要 8 字节有符号就用 `i64`,需要"与指针同宽"就用 `isize`。

Q 2. 下面的 C 代码在 Rust 里怎么写? `int x; printf("%d", x);`

✅ 参考答案

写不出来——这是 UB,编译器拒绝。必须初始化: `let x: i32 = 0; println!("{}", x);`。这正是 Rust 的设计:把 C 里"未初始化 → 垃圾值 → 难以排查"的整类 bug 变成编译错误。

Q 3. `300u16 as u8` 的结果是多少?为什么?C 里 `(char)300` 呢?

✅ 参考答案

44。 `u8` 只能存 0~255, `as` 直接截断高位:300 的二进制去掉高 8 位剩下 44。C 的 `(char)300` 行为与之一致(实现定义,实际相同),但 C 的隐式赋值 `char c = 300;` 也只给警告;Rust 则要求你显式写 `as`,把截断变成可见的决策。

Q 4. "你好, Rust" 的 `len()` 与 `chars().count()` 分别是多少?为什么?

✅ 参考答案

若用全角逗号: $3+3+3+4 = 13$ 字节,7 个字符。每个汉字 3 字节,逗号 3 字节,"Rust" 4 字节;字符数则是 $2+1+4 = 7$ 。若用 ASCII 逗号则是 11 字节。要点:`len()` 永远是字节数,与字符数无关。

Q 5. 为什么 Rust 禁止隐式类型转换?这与它的安全目标有什么关系?

✅ 参考答案

隐式转换(尤其截断、有符号↔无符号、浮点精度丢失)是 C 里大量隐蔽 bug 的来源。Rust 要求显式转换,使"类型不匹配"在编译期暴露、让程序员对每次转换负责;与"消除未定义行为"的整体目标一致——能编译错误解决的事,绝不留到运行期。

Q 6. `let x = 5; x = 6;` 能编译吗? `let mut x = 5; let x = 6;` 呢?

✅ 参考答案

第一句编译错误: `x` 绑定默认不可变,不能赋值。第二句可以:先声明可变的 `x=5`,再用 `let` 创建同名新绑定 `x=6`(遮蔽),旧绑定被隐藏。遮蔽不修改原绑定,而是创建新绑定——两者的语义完全不同。

章末实验题:学生信息登记与成绩统计

实验 1:学生信息登记与成绩统计

实验目标:用全章涉及的类型(整数/浮点/布尔/字符/字符串/数组/Vec/元组/枚举/常量/转换/格式化输出)实现一个学生成绩登记统计程序。

实验要求:

- 任务 1:定义年级枚举 Year(覆盖知识点:强类型枚举)与常量(及格线、班级容量)(覆盖:const)
- 任务 2:用 (String, Year, f64, bool) 元组 + Vec 登记不少于 5 名学生(覆盖:元组、Vec、String、浮点、布尔)
- 任务 3:统计总人数、总分、平均分、及格人数与及格率,注意类型转换(覆盖:usize→f64 的 as 转换、算术运算)
- 任务 4:给每名学生打等级 A~F,并标记是否重修,用格式化输出打印成绩单(覆盖:char、if 表达式、format 格式化、遮蔽)
- 任务 5:用字符串拼接生成班级评价语(覆盖:push_str、String)
- 任务 6:运行程序并核对输出与手工计算一致(覆盖:整体验证)

实验环境:装有 Rust 工具链(cargo --version 验证),创建项目 cargo new grade_book,进入目录后 cargo run 运行。

第一步 **一步:** cargo new grade_book 创建项目,把代码写进 src/main.rs

第二步 **二步:**先定义 Year 枚举与两个常量;注意枚举变体命名用驼峰

第三步 **三步:**用 Vec::new() 创建学生列表,push 进 5 名学生;元组字段按 (姓名, 年级, 成绩, 重修) 排列

第四步 **四步:**遍历累加时,先想清楚 sum 的类型;average = sum / total as f64 为什么 total 要转换?

第五步 **五步:**等级判定用 if 表达式链(结果赋给 char 变量),注意每个 if 分支都要返回值

第六步 **六步:**最后核对输出;尝试把及格线改成 80 分,确认输出随之变化

参考答案要点

```
// ===== 实验 1:学生信息登记与成绩统计 =====
// 覆盖知识点:常量(const)、强类型枚举(enum)、元组、Vec、
//          整数/浮点/布尔/字符/字符串、类型转换(as)、格式化输出、遮蔽

const CLASS_SIZE: usize = 5;    // 班级人数(常量)
const PASS_LINE: f64 = 60.0;    // 及格分数线(常量)

/// 学生年级:用强类型枚举代替 C 的 int 魔法数字
enum Year {
    Freshman,    // 大一
    Sophomore,   // 大二
    Junior,       // 大三
    Senior,       // 大四
}

fn main() {
    // 用元组表示一名学生:(姓名, 年级, 成绩, 是否重修)
    let mut students: Vec<(String, Year, f64, bool)> = Vec::new();

    // 登记学生:字符串、枚举、浮点、布尔字面量全覆盖
    students.push((String::from("张三"), Year::Freshman, 88.5, false));
    students.push((String::from("李四"), Year::Sophomore, 59.0, true));
    students.push((String::from("王五"), Year::Junior, 91.0, false));
    students.push((String::from("赵六"), Year::Senior, 45.5, true));
    students.push((String::from("钱七"), Year::Freshman, 73.0, false));

    // ---- 统计:总数、总分、平均分、及格人数 ----
    let total = students.len();           // usize 类型
    let mut sum: f64 = 0.0;
    let mut passed: u32 = 0;              // 无符号整数计数
    for s in &students {
        sum += s.2;                       // 元组下标访问
        if s.2 >= PASS_LINE {
            passed += 1;
        }
    }
    let avg: f64 = sum / total as f64;    // 显式转换:usize -> f64

    // ---- 打印成绩单 ----
    println!("===== 成绩单 =====");
    for s in &students {
        let score = s.2;
        // if 是表达式:每个分支返回一个 char
        let mark: char = if score >= 90.0 { 'A' }
            else if score >= 80.0 { 'B' }
            else if score >= 70.0 { 'C' }
            else if score >= 60.0 { 'D' }
            else { 'F' };
        let result = if s.3 { "(重修)" } else { "" };
        // {:<8} 左对齐占 8 宽;{:5.1} 总宽 5、保留 1 位小数
        println!("{:<8} 成绩 {:5.1} 等级 {} {}", s.0, score, mark, result);
    }
    println!("=====");
    println!("总人数:{total} 平均分:{avg:.1} 及格人数:{passed}");
}
```

```

// ---- 及格率:u32 -> f64 的转换 ----
let pass_rate: f64 = passed as f64 / total as f64 * 100.0;
println!("及格率:{:.1}%", pass_rate);

// ---- 字符串拼接生成班级评价 ----
let mut summary = String::from("班级评价:");
if avg >= 85.0 {
    summary.push_str("优秀");
} else if avg >= 70.0 {
    summary.push_str("良好");
} else {
    summary.push_str("需要努力");
}
println!("{summary}");

// ---- 补充:字符与字节字面量 ----
let first_char: char = '张';
let byte_a: u8 = b'A';
println!("首字符 {first_char},ASCII 码 {byte_a}");
let _ = CLASS_SIZE; // 避免未使用警告(教学示例)
}

```

设计说明:程序刻意把"每个类型"都用一次——常量管配置、枚举管类别、元组管打包、Vec 管动态增长、as 管单位换算、if 表达式管分支取值、format 语法管对齐输出。运行后把及格线改为 80,观察及格人数与评价语的变化,即可验证各任务覆盖的知识点确实在协同工作。

下一章预告:第2章 运算符与表达式——Rust 没有 ++ 与 --,溢出在 debug 模式直接 panic;if/loop/match 都是表达式,还有 C 里没有的 ? 运算符与范围运算符 ..。类型转换的坑刚过,运算的坑马上来。